



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Lifelong Adaptation in Minimal Structures: A Spiking-NEAT Architecture with R-STDP

Estudiante: Naiara Manteiga Cendón

Dirección: Outro Nome Completo

A Coruña, October 2026.

Dedicatoria

I would like to thank ...

Acknowledgements

I would like to thank ...

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Resumo

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Keywords:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Palabras chave:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.2.1	Main objective	2
1.2.2	Specific Objectives	3
1.3	Chosen approach	3
2	Theoretical Background	5
2.1	Neuroevolution and Genetic Algorithms	5
2.1.1	TWEANNs	6
2.1.2	NeuroEvolution of Augmented Topologies (NEAT)	9
2.1.2.1	Innovation Numbers	9
2.1.2.2	Speciation	10
2.1.2.3	Minimal structures	11
2.1.2.4	Why is it better	11
2.1.2.5	Limitations	12
2.2	Green AI and Spiking Neural Networks (SNNs)	13
2.2.1	Green AI	13
2.2.2	SNN Fundamentals and the LIF Model	14
2.2.3	Training Spiking Networks	15
2.2.4	Theoretical Energy Estimation	15
2.3	Synaptic Plasticity and Biological Reinforcement Learning	16
2.3.1	Spike-Timing-Dependent Plasticity (STDP)	16
2.3.2	Reward-Modulated STDP (R-STDP)	17
3	State of the Art	18
3.1	Game-playing controllers	18
3.2	Spiking networks for control	19

3.3	Reward-modulated plasticity in spiking agents	20
3.4	Evolved plastic networks	20
3.5	Energy-aware evaluation and neuromorphic hardware	21
3.6	Comparative analysis and research gap	21
4	Conclusións	23
	Bibliography	25

List of Figures

2.1	Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks.	6
2.2	The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings. Adapted from [1].	7
2.3	Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there is a preset chance that an inherited gene is disabled if it is disabled in either parent. Adapted from [1].	8

List of Tables

3.1	Positioning of related work. “n/r”: not reported in the cited work. “Local plasticity” means that weights change through local (STDP-type) rules; it does not indicate whether learning continues after deployment.	22
-----	---	----

Introduction

1.1 Motivation

Artificial Neural Networks (ANNs) have been at the forefront of computer science research for the past few decades, the core concept of building artificial systems that simulate the biological behavior of the human brain remains a prominent research focus. In recent years, the capabilities of Deep Learning and transformer architectures have grown substantially, driven by their remarkable results and the commercialization of General Purpose Artificial Intelligence (GPAI). Nonetheless, these models suffer from two major shortcomings: their unsustainable energy and environmental consumption, and the massive initial training datasets they require. This severe energetic cost has given rise to initiatives such as Green AI [5], which advocates for computational efficiency as a primary evaluation criterion alongside model accuracy. Conversely, neuroevolutionary algorithms attempt to replicate both the mechanics of the brain and the evolutionary processes of living organisms. In particular, the NeuroEvolution of Augmented Topologies (NEAT) algorithm [1] offers a structural advancement from its predecessors, yielding excellent results for continuous, high-dimensional control tasks, as will be detailed in subsequent sections.

The practical application of this project is heavily inspired by a well-known demonstration of the NEAT algorithm navigating a Super Mario Bros. environment, an implementation its creator named MarI/O [6]. This demonstration clearly illustrates that NEAT does not require massive training datasets nor any prior information about the environment. MarI/O begins the simulation without knowing the controls, discovers movement through trial and error, and evolves until it can clear the level autonomously. This effectively solves the first major shortcoming regarding initial data dependency.

However, standard NEAT does not resolve the energy consumption problem on its own. To address this, this project aims to implement two biologically inspired principles: Spiking Neural Networks (SNNs) [2] and Reward-Modulated Spike-Timing-Dependent Plasticity (R-

STDP) [4, 8]. SNNs are built upon the premise that biological brains communicate via discrete energy "spikes". To put this into perspective, standard ANNs operate like an open tap where water (energy) flows continuously and without interruption; in contrast, adapting SNNs is akin to closing the tap to a bare minimum, allowing water to accumulate and only fall as a single drop once it reaches a specific density. Furthermore, the integration of R-STDP aims to enhance the evolutionary process. While standard NEAT evolves both topology and initial weights across generations, R-STDP focuses on dynamically evolving the synaptic weights during gameplay. This dual mechanism grants the agent greater adaptability, theoretically allowing the "brain" to generalize to new levels or similar tasks without the need to restart the training process from scratch.

The practical relevance of this architecture extends to constrained engineering environments. A rover or search-and-rescue drone could use local R-STDP to adapt after a component failure without relying on a remote server. Remote Internet-of-Things sensors could remain dormant and spike only when an acoustic or environmental anomaly is detected. Medical devices such as neural implants or smart pacemakers could process changing biological signals locally, while wearable devices could personalize their models without continuously transmitting private data to the cloud.

Ultimately, the exploration of SNNs and R-STDP in this project aligns well with the cutting edge of neuromorphic engineering. As modern technology increasingly shifts toward edge computing, autonomous robotics, and Internet of Things (IoT) devices, the demand for AI systems capable of real-time adaptation with minimal power consumption is increasing. Therefore, validating these biological learning mechanisms within a dynamic software environment like Super Mario Bros not only addresses an immediate academic gap, but also directly contributes to the foundation of next generation, hardware deployable AI.

1.2 Objectives

1.2.1 Main objective

The primary objective of this project is to develop an autonomous Artificial Intelligence program, commonly referred to as an 'agent', capable of navigating and completing levels within the Super Mario Bros environment. In this context, an agent is an independent computational entity that continuously observes its surroundings and makes decisions to achieve a specific goal. This decision-making process will be driven by Reinforcement Learning, a machine learning paradigm in which the agent learns optimal behaviors through trial, error, and the reception of environmental rewards.

To achieve this while addressing the energy inefficiencies of modern AI, the project will substitute the computationally expensive continuous neural networks traditionally used in

these tasks with a biologically plausible Spiking Neural Network (SNN). Furthermore, the agent’s learning mechanism will be governed by Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP), enabling it to dynamically adapt its synaptic connections in real-time.

Ultimately, the goal is to evaluate whether an agent built upon these biologically inspired principles can maintain adaptability in a dynamic environment while theoretically reducing energy consumption.

The experimental environment will be the Super Mario Bros implementation provided by the gym-super-mario-bros package [12].

For this document, comparable performance is defined operationally: the spiking agent should reach at least 90% of the baseline mean distance and completion rate, while requiring no more than 10% additional time to complete the level. These metrics will be reported as mean and standard deviation over five randomized seeds on Level 1-1. Training energy is outside the primary energy metric; the total number of evaluated frames will nevertheless be reported for transparency.

1.2.2 Specific Objectives

To achieve the main objective, the project is divided into the following sub-goals:

- **Establish a Baseline:** Implement and train a standard NEAT algorithm to play Super Mario Bros to serve as a performance and architectural baseline.
- **Implement Spiking Mechanics:** Replace standard continuous activation functions (e.g., Sigmoid) with a discrete Leaky Integrate-and-Fire (LIF) [2] neuron model and establish a spike-encoding method for environment inputs.
- **Integrate Local Plasticity:** Incorporate Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP) to allow the agent’s synaptic weights to adapt dynamically based on environmental rewards.
- **Evaluate Efficiency:** Compare the performance and theoretical computational energy consumption of the Spiking-NEAT agent against the standard NEAT baseline.

1.3 Chosen approach

To achieve the expected results, the project methodology is conceptually divided into three metaphorical pillars: the architect, the engine, and the teacher.

The Architect (Topology): is the foundational structure of the agent’s brain is generated using NeuroEvolution of Augmented Topologies (NEAT). NEAT improves upon earlier

Topology and Weight Evolving Artificial Neural Networks (TWEANNs) by utilizing historical markings (innovation numbers), speciation, and incremental growth from minimal structures. Because NEAT has been validated on complex control tasks like double pole balancing, it serves as the ideal "architect" to dynamically grow the network topology for this project.

The Engine (Activation Function): Standard NEAT typically employs a continuous sigmoidal activation function bounded between $[0, 1]$. However, because these activation values are continuous floating-point fractions, the underlying hardware is forced to perform computationally expensive Multiply Accumulate (MAC) operations for every single synapse, at every single time step. Even if a node's activation is microscopically close to zero, the system must still execute floating-point arithmetic for every active synapse to calculate the next frame. This persistent, unconditional computation is exactly what drives the massive energy consumption problem acknowledged in the previous section. To resolve this, the network will be transformed into a Spiking Neural Network (SNN) by implementing the Leaky Integrate-and-Fire (LIF) model [2]. The LIF model is a mathematical abstraction of biological neurons focusing entirely on the timing of electrical spikes via three mechanics: integrate (accumulating incoming electrical signals), leak (the natural decay of voltage over time), and fire (emitting a binary spike when the voltage surpasses a threshold and resetting). Expanding on the previous water tap metaphor, the LIF model acts as a bucket with a small hole that fills up until the bucket is full and gets emptied (resets) after. The tap represents synaptic current (integrate), the water level represents membrane potential, the hole represents natural leakage (leak), and the overflow when the bucket fills up completely represents the spike (fire). This discrete, binary event-driven approach is highly efficient because neurons only perform calculations when a discrete spike occurs, essentially reducing continuous matrix multiplication to sparse, binary addition.

The Teacher (Learning Rule): In standard NEAT, synaptic weights are mutated randomly between generations, which can result in inefficient learning. To address this, the project introduces Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP) as the "teacher". R-STDP acts as a biologically plausible reinforcement learning mechanism. Rather than randomly guessing weight values, R-STDP analyzes which neurons spiked immediately prior to a successful in-game action and strengthens those specific connections. By mimicking the release of dopamine in a biological brain, the model is directly encouraged to repeat productive behaviors.

NEAT evolves the topology and the initial weights, while R-STDP adapts the effective weights during each episode. Learned weights are reset to the genome-encoded initial values at the beginning of a new episode rather than being written back to the genome. Several episodes per generation will therefore be evaluated so that lifetime adaptation can influence fitness; Lamarckian write-back may be evaluated separately as an ablation.

Theoretical Background

This chapter establishes the theoretical and mathematical foundations required to contextualize the contributions of this document. It details the mathematical models and algorithms surrounding neuroevolutionary algorithms, Spiking Neural Networks (SNNs), and biologically plausible learning rules. By analyzing recent advancements and their inherent limitations, this section constructs the conceptual framework necessary to justify the proposed Spiking-NEAT architecture and its application in dynamic control environments.

2.1 Neuroevolution and Genetic Algorithms

Historically, the standard method for training Artificial Neural Networks (ANNs) is the backpropagation algorithm. The fundamental purpose of backpropagation is to optimize a network's predictions through supervised learning by systematically adjusting its internal weights and biases to minimize a mathematical error.

As illustrated in Figure 2.1, this training process operates in four distinct phases:

1. **The Forward Pass:** Input data propagates forward through the network's interconnected hidden layers. Each node extracts features and applies continuous mathematical transformations until the final layer outputs a prediction.
2. **Error Calculation:** A predefined loss function evaluates this output, measuring the mathematical discrepancy (the error) between the network's prediction and the actual "ground truth" provided by the training dataset.
3. **The Backward Pass:** The algorithm calculates the gradient of the loss function. It then moves backward through the network, effectively distributing the "blame" for the error by calculating exactly how much each individual weight and bias needs to be adjusted to improve the next prediction.

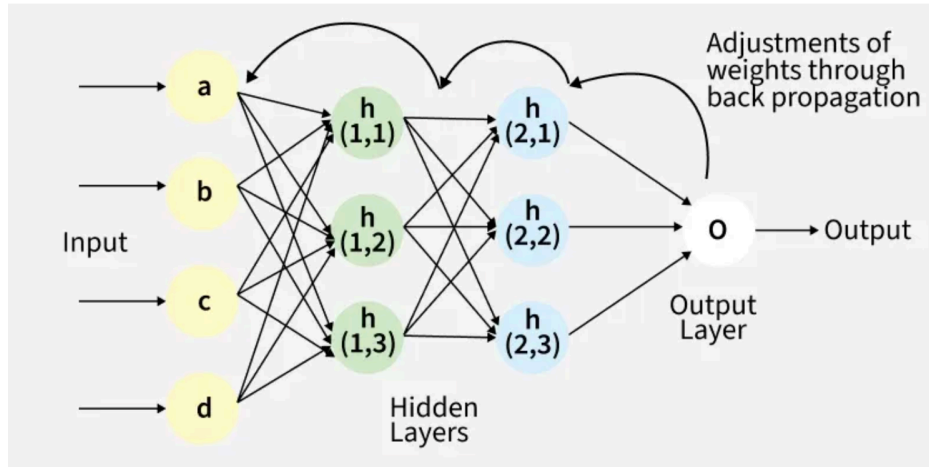


Figure 2.1: Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks.

4. **The Optimizer Step:** An optimizer uses the calculated gradients to update the network weights and improve the next prediction.

While this gradient descent method is incredibly powerful for supervised tasks like image classification, it possesses two strict requirements: it relies on massive datasets of “correct answers” to calculate the initial loss, and it requires continuous, differentiable activation functions to calculate the gradients.

However, in dynamic control environments (such as autonomous robotics or video games like Super Mario Bros) there is no dataset of “perfect moves” to learn from. The AI only knows whether it survived or failed based on sparse environmental rewards. Because standard gradient-based backpropagation struggles in these unpredictable, data-poor scenarios, researchers turned to biology for a solution, giving rise to Neuroevolution (NE).

Neuroevolution is a prominent subfield of Artificial Intelligence that applies the principles of Darwinian natural selection to neural networks. Instead of training a single network with a dataset, Neuroevolution relies on Genetic Algorithms (GAs) to generate a “population” of many different candidate networks. The algorithm evaluates each network based on its performance in the environment (its fitness). The weakest networks are discarded, while the most successful ones are selected to produce a new generation through biological mechanics like crossover (combining traits from two parents) and mutation (random structural changes).

2.1.1 TWEANNs

Historically, early neuroevolutionary systems focused exclusively on optimizing the synaptic weights of a neural network, while its structural topology remained fixed by a human en-

gineer. However, guessing the optimal architecture for a complex, dynamic environment is incredibly difficult. To overcome this, researchers developed Topology and Weight Evolving Artificial Neural Networks (TWEANNs) [1], algorithms designed to evolve both the connection weights and the physical structure of the network simultaneously.

While TWEANNs represented a significant conceptual leap, their practical application was severely hindered by three critical limitations:

1. **The Competing Conventions (Permutation) Problem:** This is the most severe flaw in early neuroevolution. It occurs when two parent networks learn to solve the same problem, but encode their solutions using different internal topological orderings. If a genetic algorithm attempts to cross over (breed) these two parents, the resulting offspring will often suffer from missing or duplicated features. Because the algorithm has no way to align the mismatched hidden nodes, crossing over two highly successful parents frequently produces damaged offspring.

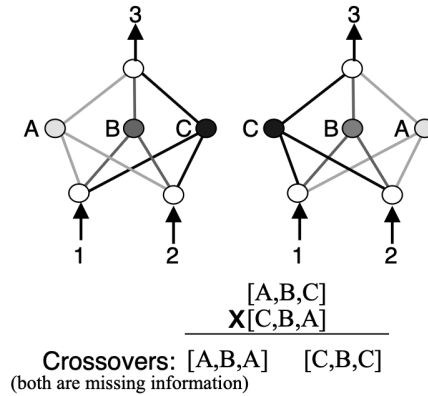


Figure 1: The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings.

Figure 2.2: The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings. Adapted from [1].

As illustrated in Figure 2.2, crossing over two functional networks with different internal orderings results in offspring missing crucial information (e.g., missing node C or node A).

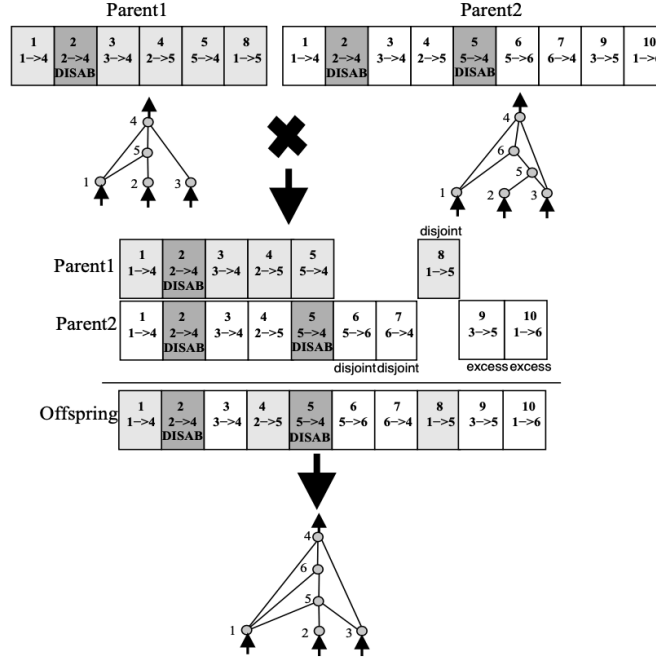


Figure 4: Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there's a preset chance that an inherited gene is disabled if it is disabled in either parent.

Figure 2.3: Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there is a preset chance that an inherited gene is disabled if it is disabled in either parent. Adapted from [1].

2. **Unprotected Innovation:** In evolutionary algorithms, a newly mutated structure (like a new hidden node) often causes a temporary drop in performance before its weights can be optimized. Early TWEANNs evaluated all networks in a single, unified pool. Consequently, new and complex topological innovations were immediately driven to extinction by older, structurally simpler networks that had already optimized their weights. Also, having the global population all together can cause the initially fitter individuals to prevail and drive other species of the global population to extinction; this is a hazard for innovation.
3. **Topological Bloat:** To ensure genetic diversity, early models frequently initialized populations with complex, highly randomized topologies. This bloated the genomes with unjustified connections from the very first generation, unnecessarily expanding the mathematical search space and wasting computational resources.

These three structural flaws meant that TWEANNs struggled to scale to high-dimensional tasks. The field required a structural advancement to make neuroevolution mathematically efficient and viable for complex control environments, a shift that was realized by the introduction of the NEAT algorithm.

2.1.2 NeuroEvolution of Augmented Topologies (NEAT)

NeuroEvolution of Augmented Topologies (NEAT) [1] is a highly effective algorithm that resolves the historical limitations of TWEANNs. As detailed by Stanley and Miikkulainen [1], NEAT achieves increased efficiency through a principled method of crossover via historical markings, protection of structural innovation using speciation, and incremental growth from minimal structures. These mechanisms allow evolution to optimize and complexify solutions simultaneously, providing a powerful foundation for dynamic control tasks.

2.1.2.1 Innovation Numbers

The fundamental mechanism that allows NEAT to resolve the Competing Conventions problem is the implementation of historical markings, or innovation numbers. In biological evolution, homology allows organisms with different genetic structures to cross over safely because their genes share a common historical origin. NEAT mathematically replicates this biological principle by assigning a unique, incrementally increasing integer, a global innovation number, to every new structural mutation (such as a new connection or a new hidden node) that occurs within the population.

This historical tracking transforms the complex topological analysis required by earlier TWEANNs into a highly efficient linear alignment problem. When two parent networks are selected for crossover, their genomes are aligned side-by-side using these numerical IDs.

To perform this crossover without losing information, the algorithm categorizes the aligned genes into three distinct types:

- **Matching genes:** Genes with the exact same innovation number present in both parents. These genes are inherited randomly by the offspring.
- **Disjoint genes:** Genes present in one parent but missing in the other, while still falling strictly within the numerical range of the other parent's innovation numbers.
- **Excess genes:** Genes present in one parent but missing in the other. These genes do not match at the end of the genome sequence, falling outside the numerical range of the overlapping innovation numbers.

During the recombination process, disjoint and excess genes are systematically inherited from the more fit parent to ensure advantageous structures are preserved. If both parents share equal fitness levels, these disjoint and excess genes are instead inherited randomly. Furthermore, to preserve structural memory, there is a preset probability that an inherited gene will remain disabled in the offspring if it was disabled in either of the parent genomes.

By utilizing these historical markings, NEAT ensures that crossover remains orderly. The algorithm can effectively recombine networks of vastly different sizes and topologies without losing track of which gene performs which function, effectively ensuring that structural innovations are safely transferred to the next generation without information loss.

2.1.2.2 Speciation

The second major mechanism NEAT introduces to stabilize the evolutionary process is speciation, also known as niching. In traditional neuroevolution, adding a new structural mutation (such as a new node or connection) often causes a temporary drop in the network's overall performance before its new weights can be properly optimized. If the entire population competes in a single, unified pool, the initially fitter individuals prevail, and these new, innovative topologies are quickly driven to extinction by older, structurally simpler networks that have already fine-tuned their synaptic weights.

To prevent this hazard to innovation, NEAT divides the global population into distinct species based on their topological similarity. This categorization is achieved by comparing the historical innovation numbers discussed in the previous section. Genomes with a high number of matching genes are grouped together into the same species, while those with many disjoint or excess genes are separated into different niches.

Once the population is divided, NEAT applies explicit fitness sharing to regulate competition and protect diversity. This mechanism forces individuals with similar genomes to share their fitness payoff within their specific niche, effectively utilizing fitness peaks to balance

the population. By distributing the fitness rewards in this manner, the algorithm ensures that species compete primarily within their own topological group rather than against the entire global population. Consequently, initially "weaker" species are protected from premature extinction, granting newly mutated structures the necessary time and generational cycles to optimize their weights and demonstrate their true functional potential.

2.1.2.3 Minimal structures

The third core innovation of NEAT addresses how the evolutionary process begins, specifically resolving the initialization problem observed in earlier Topology and Weight Evolving Artificial Neural Networks (TWEANNs). Historically, early TWEANNs initialized their populations with complex, highly randomized topologies to ensure genetic diversity. However, this approach inherently bloated the genomes with unjustified connections from the very first generation, which unnecessarily expanded the mathematical search space and wasted computational resources.

NEAT fundamentally reverses this paradigm by starting the population with a completely minimal structure, meaning the initial candidate networks contain zero hidden nodes. Instead of beginning with complexity, NEAT allows structural complexity to emerge incrementally over time. New hidden nodes and synaptic connections are only added to the network through mutations if they provide a distinct functional benefit to the agent's performance.

By incrementally growing from minimal structures, NEAT ensures that the final optimized network remains compact. This strategy drastically minimizes the dimensionality of the search space, which significantly boosts the algorithm's overall performance. Furthermore, this approach represents an important contribution to Genetic Algorithms (GAs) because it demonstrates that evolution can simultaneously optimize and complexify solutions. It offers the possibility of evolving increasingly complex architectures over generations, strongly reinforcing the algorithm's analogy with biological evolution.

2.1.2.4 Why is it better

Usually, the standard method for training Artificial Neural Networks (ANNs) is the backpropagation algorithm, which relies heavily on massive datasets of "correct answers" to calculate initial loss and requires continuous, differentiable activation functions to compute gradients. While Deep Reinforcement Learning successfully uses backpropagation for control in environments like Super Mario Bros, it requires massive amounts of environment interactions. Neuroevolution provides a gradient-free alternative. In these unpredictable scenarios, there is no dataset of "perfect moves" to learn from, the agent only knows whether it survived or failed based on sparse environmental rewards.

Neuroevolution, and NEAT in particular, presents a viable alternative for these data-poor environments. Through extensive ablation studies and benchmark testing on complex control tasks (such as the XOR problem and double pole balancing) NEAT has consistently demonstrated superior performance over traditional neuroevolutionary methods. By leveraging its three core mechanisms (innovation numbers, speciation, and incremental growth from minimal structures), NEAT systematically resolves the Competing Conventions problem, protects structural innovation, and prevents topological bloat, thereby keeping the mathematical search space highly efficient.

Consequently, NEAT was chosen as the foundational "architect" to dynamically grow the network topology for this project. Its practical application is heavily inspired by Marl/O, a well-known demonstration of the NEAT algorithm successfully navigating a Super Mario Bros environment. This demonstration clearly illustrates that NEAT requires absolutely no prior information about the environment nor massive training datasets to function. The agent begins the simulation completely unaware of the controls, discovers movement through trial and error, and evolves purely based on fitness evaluation until it can clear the level autonomously. Because NEAT effectively solves the major shortcoming regarding initial data dependency and yields excellent results for continuous, high-dimensional control tasks, it serves as the effective baseline architecture for the subsequent integration of Spiking Neural Networks and R-STDP.

2.1.2.5 Limitations

Despite this structural efficiency and proven success, current standard NEAT architectures possess a structural limitation: they rely heavily on continuous mathematical activation functions, such as the Sigmoid function, which are bounded between $[0, 1]$. Because these activation values are continuous floating-point fractions, the underlying hardware is forced to perform computationally expensive Multiply Accumulate (MAC) operations for every single synapse, at every single time step. Even if a node's activation is microscopically close to zero, the system must still execute floating-point arithmetic for every active synapse simply to calculate the next frame.

This persistent, unconditional computation drives a massive energy consumption problem. Consequently, standard NEAT becomes computationally demanding for modern edge-computing scenarios, such as autonomous robotics, aerospace applications, and Internet of Things (IoT) devices, where extreme energy efficiency is mandatory. This critical computational bottleneck exposes the gap in current research and directly necessitates the transition toward Spiking Neural Networks (SNNs) to successfully address the energy constraints of next-generation, hardware-deployable AI.

2.2 Green AI and Spiking Neural Networks (SNNs)

2.2.1 Green AI

As previously mentioned, the world is currently experiencing the rapid rise and commercialization of AI and GPAI. However, there is ongoing dispute regarding the computational cost of running these machines and the vast volumes of data they require. Most of these modern architectures fall under the category of Red AI. According to Schwartz et al., “Red AI refers to AI research that seeks to obtain state-of-the-art results in accuracy (or related measures) through the use of massive computational power—essentially ‘buying’ stronger results” [5]. While Red AI focuses primarily on maximizing accuracy through sheer computational force, Green AI advocates for treating energy efficiency and computational cost as primary evaluation metrics alongside accuracy [5]. Consequently, this project evaluates energy efficiency as a core metric, operating under the premise that it is just as valuable as model accuracy.

The cost of producing an AI result can be described by the cost of one model execution, the dataset size, and the number of hyperparameter experiments:

$$\text{Cost}(R) \propto E \cdot D \cdot H. \quad (2.1)$$

Here, E is the execution cost for one example, D is the dataset size, and H is the number of experiments.

Furthermore, standard implementations are computationally expensive primarily because they rely on continuous floating-point weights and activations, requiring the underlying hardware to perform Multiply-Accumulate (MAC) operations for every active synapse at each timestep. The activation function is a mathematical equation that determines whether a given node should remain inactive or fire, defining its output. The vast majority of Red AI alternatives utilize continuous activation functions, such as Sigmoid or ReLU. The fundamental issue is that these functions rely on continuous floating-point numbers, requiring the underlying hardware to perform complex matrix floating-point operations constantly.

As Schwartz et al. observe: “Ironically, deep learning was inspired by the human brain, which is remarkably energy efficient” [5]. As mentioned previously, a biological brain does not physically operate like an open tap, rather, it processes information via discrete spikes. A biological neuron is divided into three main components: dendrites, the soma, and the axon. Roughly speaking, the dendrites can be considered the “input system,” the soma acts as the central processor or “GPU,” and the axon serves as the “output system” [2]. Electrical energy travels through these components, and a sufficient accumulation of voltage triggers an action potential, which is referred to as a discrete spike. A sequence of these spikes over time constitutes a spike train. These biological mechanics form the foundational principles of Spiking

Neural Networks (SNNs) [2]. Utilizing spikes as an activation mechanism renders the architecture significantly more energy-efficient, as it relies on discrete, binary event-driven signals rather than continuous floating-point operations. Furthermore, SNNs are fundamentally inspired by electrical circuitry, making them highly optimized for neuromorphic hardware translation.

2.2.2 SNN Fundamentals and the LIF Model

However, for an SNN to function computationally, it requires a physical mathematical abstraction. In this project, that abstraction is the Leaky Integrate-and-Fire (LIF) model [2]. As the name suggests, the LIF model governs neuronal dynamics through three distinct stages: integrate, leak, and fire. First, the neuron accumulates energy from incoming synaptic spikes, which raises its internal membrane potential (integration). Second, the membrane possesses a natural electrical resistance, if spikes do not arrive fast enough, the membrane progressively loses voltage, returning toward its resting state (leak). Finally, if the integration outpaces the leak and surpasses a strict voltage threshold, the neuron instantly emits a binary spike and resets its internal voltage to a baseline (fire). In this architecture, every continuous node generated by the NEAT algorithm is replaced by this discrete LIF physical model.

The LIF model can be written as an electrical circuit with membrane capacitance C , membrane resistance R , input current $I(t)$, and membrane potential $u(t)$:

$$I(t) = I_R(t) + I_C(t), \quad I_R(t) = \frac{u(t) - u_{\text{rest}}}{R}, \quad I_C(t) = C \frac{du(t)}{dt}. \quad (2.2)$$

Combining these terms gives the membrane equation

$$\tau_m \frac{du(t)}{dt} = -(u(t) - u_{\text{rest}}) + RI(t), \quad \tau_m = RC. \quad (2.3)$$

When $u(t)$ reaches the threshold ϑ , the neuron emits a spike and its potential is reset to u_{reset} .

This section has outlined the significant computational advantages of SNNs and the LIF model, demonstrating why they serve as a powerful framework for achieving Green AI efficiency. While combining the NEAT algorithm with LIF neurons yields an proposed Spiking-NEAT architecture, it introduces a critical challenge: this discrete, non-differentiable implementation are difficult to train natively with standard backpropagation due to non-differentiable spikes, often requiring workarounds like surrogate gradients. A Spiking-NEAT architecture requires a biologically plausible learning rule to optimize its weights, which leads to the final core component of this project: biological Reinforcement Learning.

2.2.3 Training Spiking Networks

Historically, SNNs were viewed as difficult to train via standard backpropagation due to the non-differentiable nature of discrete spikes. While modern techniques like Surrogate Gradients [9] have emerged to approximate these derivatives and enable gradient-based optimization in SNNs, they can be memory-intensive for continuous, online learning. Therefore, for an autonomous agent requiring real-time adaptability, biologically plausible local learning rules like Spike-Timing-Dependent Plasticity (STDP) present an attractive alternative.

Furthermore, integrating SNNs with neuroevolution provides a framework for discovering effective network architectures. For instance, Qiu et al. [3] demonstrated that recurrent spiking controllers evolved using NEAT can be applied to continuous nonlinear control domains, such as the pole-balancing problem. While Qiu et al. utilized NEAT to evolve the static weights and topologies of SNNs, the incorporation of local learning rules like R-STDP for intra-generational weight adaptation remains under-explored.

2.2.4 Theoretical Energy Estimation

The energy comparison in this project is limited to inference-time arithmetic operations. A conventional controller is modelled using Multiply-Accumulate (MAC) operations, while the spiking controller is modelled using Synaptic Operations (SynOps), membrane updates, and input encoding:

$$E_{\text{ANN}} = (N_{\text{MAC}} E_{\text{MAC}}) + (N_{\text{activation}} E_{\text{activation}}). \quad (2.4)$$

$$E_{\text{SNN}} = (N_{\text{SynOp}} E_{\text{AC}}) + (N_{\text{update}} E_{\text{update}}) + E_{\text{encoding}}. \quad (2.5)$$

Using the commonly cited 45-nm estimates from Horowitz [13], a 32-bit floating-point MAC costs approximately 4.6 pJ and an accumulate operation costs approximately 0.9 pJ. Membrane leak updates are counted explicitly rather than ignored, and memory-access energy is outside the scope of this arithmetic model. The reported efficiency metric is therefore the ratio $E_{\text{SNN}}/E_{\text{ANN}}$, together with the operation counts used to obtain it.

In the SNN, N_{update} represents the total number of membrane leak updates. Because biological neurons continuously “leak” voltage over time, calculating this natural decay every frame requires a decimal multiplication; therefore, E_{update} is treated as equivalent to a heavy MAC operation (4.6 pJ), preventing an underestimation of the SNN cost. Furthermore, because SNNs process discrete spikes rather than raw pixels, the extra mathematical operations needed to convert the observation grid into spike trains over the internal time steps are calculated as E_{encoding} . This encoding cost is obtained by counting the operations required to

convert the observation grid into spike trains, summed over the evaluation episode.

This model explicitly focuses on arithmetic processing and ignores memory-access costs, even though data movement between RAM and the processor typically consumes more energy than arithmetic. The final ratio $E_{\text{SNN}}/E_{\text{ANN}}$ is therefore the primary metric for the inference energy budget, accompanied by the operation counts and assumptions used to compute it.

2.3 Synaptic Plasticity and Biological Reinforcement Learning

2.3.1 Spike-Timing-Dependent Plasticity (STDP)

Most of the conventional AI architectures discussed previously rely on continuous activation functions and smooth, differentiable curves, therefore, the standard method for training these Artificial Neural Networks (ANNs) is Backpropagation. Because Backpropagation calculates gradients using loss and error functions, it strictly requires continuous values. In contrast, the Spiking-NEAT approach utilizes non-differentiable, discrete binary signals (spikes). Consequently, traditional gradient-descent methods cannot be applied. Furthermore, relying solely on NEAT’s random weight mutations between evolutionary generations is computationally slow and inefficient. Therefore, this project proposes an alternative, biologically plausible solution to dynamically evolve the synaptic weights while the agent interacts with the environment.

In biological brains, learning is driven by synaptic plasticity, which refers to the ability of neural connections to strengthen or weaken over time based on their activity. This correlates with the Hebbian postulate: “Neurons that fire together, wire together.” This rule states that if neuron A repeatedly and persistently takes part in firing neuron B, the synaptic efficacy from A to B increases [2, 7]. These principles are mathematically formalized in Spike-Timing-Dependent Plasticity (STDP). In STDP, the change in synaptic weight depends heavily on the precise timing of spikes. Generally, if a pre-synaptic neuron fires just before a post-synaptic neuron, the connection between them is strengthened, a process known as Long-Term Potentiation (LTP). Conversely, if the pre-synaptic neuron fires just after the post-synaptic neuron, the connection is weakened, resulting in Long-Term Depression (LTD).

For a spike-time difference $s = t_{\text{post}} - t_{\text{pre}}$, a standard pair-based STDP learning window is

$$W(s) = \begin{cases} A_+ \exp(-s/\tau_+), & s > 0, \\ -A_- \exp(s/\tau_-), & s < 0, \end{cases} \quad (2.6)$$

where A_+ and A_- control potentiation and depression, while τ_+ and τ_- determine their

temporal ranges.

However, standard STDP is strictly unsupervised, it evaluates only the relative timing of spikes, lacking any mechanism to determine whether the resulting action was beneficial or detrimental to the system. Consequently, if a negative action occurs repeatedly, STDP will continually strengthen the synaptic pathways responsible for that action simply because the neurons fire in sequence. Without a goal-oriented mechanism, standard STDP can reinforce maladaptive behaviors, ultimately hindering the agent's progression in dynamic environments.

2.3.2 Reward-Modulated STDP (R-STDP)

To resolve this credit assignment problem, Reward-Modulated STDP (R-STDP) is introduced [4, 8, 9, 10, 11]. R-STDP integrates the principles of Reinforcement Learning by expanding standard STDP into a three-factor learning rule. The three key factors are: the pre-synaptic spike, the post-synaptic spike, and a delayed neuromodulatory signal, which mathematically mimics the release of dopamine in biological brains. In this model, when neurons fire sequentially, STDP initially marks the connection with a temporary "eligibility trace." If the agent subsequently performs a beneficial action and receives an environmental reward (a dopamine boost) shortly after, this trace is converted into a permanent weight increase (LTP). Conversely, if the agent receives a punishment or no reward, the connection is actively weakened (LTD) [4]. Consequently, R-STDP not only evolves the network weights dynamically but also ensures that learning is goal-directed and efficient.

The eligibility trace and reward-modulated weight update can be expressed as

$$\frac{de_{ij}(t)}{dt} = -\frac{e_{ij}(t)}{\tau_e} + F_{ij}(t), \quad \frac{dw_{ij}(t)}{dt} = d(t)e_{ij}(t), \quad (2.7)$$

where $F_{ij}(t)$ is the instantaneous pre-post STDP contribution, τ_e is the trace time constant, and

$$d(t) = R(t) - \langle R \rangle \quad (2.8)$$

is the neuromodulatory prediction error. Positive values strengthen tagged synapses, while negative values weaken them.

In conclusion, this triad forms the complete theoretical foundation of the proposed model: NEAT acts as the architect to build the minimal network topology, LIF neurons serve as the highly energy-efficient processing engine using discrete spikes, and R-STDP functions as the biological reinforcement learning mechanism, adjusting synaptic weights dynamically based on behavioral success rather than isolated timing.

State of the Art

This chapter positions the proposed Spiking-NEAT controller with R-STDP against existing work. Because the project draws on several research communities, the review is organised around four questions: how game-playing controllers are learned (with or without gradients), what computational substrate they use (conventional or spiking networks), whether they use local plasticity, and whether their energy cost is analysed. Sections 3.1 to 3.5 review each area, and Section 3.6 compares the works and states the research gap.

3.1 Game-playing controllers

Arcade games and platformers are standard testbeds for learning control from interaction. The Arcade Learning Environment [14] offers a common interface to Atari 2600 games, and the Mario AI Benchmark [15], built around a Super Mario Bros.-style platformer, was used in a series of game-AI competitions.

Gradient-based deep reinforcement learning. Deep Q-networks reached human-level performance on a large set of Atari games, learning from screen pixels and the game score alone [?]. This approach trains deep networks by backpropagation and is data-hungry: DQN, for example, was trained on 50 million frames per game. The computational cost is paid mostly during training, but the resulting controller is a dense floating-point network that is evaluated at every decision step.

Neuroevolution. Evolutionary methods optimise a population of controllers using only episode-level fitness, so they need neither gradients nor labelled data. Hausknecht et al. [16] compared four neuroevolution algorithms, among them NEAT and HyperNEAT, on 61 Atari games with different state representations. They found that direct encodings such as NEAT worked best with compact state representations, whereas the indirect encoding of Hyper-

NEAT scaled to the raw screen. Later work showed that simple genetic algorithms and evolution strategies can be competitive with gradient-based deep reinforcement learning for networks with millions of parameters [17, 18], and Stanley et al. [19] review the field.

NEAT [?] is the best-known method that evolves network structure together with weights, starting from minimal networks (Chapter 2). Its application to Super Mario World in the MarI/O demonstration [6] motivated this project. MarI/O is an informal demonstration rather than a controlled study: it is not peer reviewed and reports no baseline or energy analysis. It nevertheless shows that topology evolution can discover level-clearing behaviour from a compact tile-based observation, without demonstrations or labelled data.

Summary. The controllers above are conventional continuous-valued networks evaluated densely at every step, performance is reported as score or distance, and, to the best of our knowledge, none of these works reports inference energy.

3.2 Spiking networks for control

Gradient-based training. Spikes are not differentiable, but spiking networks can still be trained with backpropagation-style methods through surrogate gradients [20], and recurrent spiking networks can be trained with local approximations of backpropagation through time such as e-prop [21]. In control, these ideas have been combined with deep reinforcement learning. PopSAN trains a population-coded spiking actor together with a deep critic and was deployed on Intel’s Loihi chip, where it was benchmarked against conventional deep reinforcement learning algorithms on continuous-control tasks [22]. SANSAC replaces the actor of Soft Actor-Critic with a spiking network and reports near-equivalent performance to the conventional actor on standard hardware [?]. In both cases the spiking network is the inference-time actor, while learning relies on a conventional critic and gradient-based optimisation.

Evolutionary training. Qiu et al. [3] evolve a recurrent spiking controller with a topology-evolution algorithm for a classic nonlinear control problem. They propose two mechanisms for decoding continuous control signals from spikes and report that the spiking controller finds functional solutions significantly faster than sigmoidal networks. Their emphasis is on evolving structure and decoding spikes, not on learning during an episode. Pan et al. [?] evolve brain-inspired modular architectures for spiking networks with a multi-objective algorithm that also targets energy, but they evaluate on image-classification benchmarks, both static and neuromorphic, rather than on sequential control.

3.3 Reward-modulated plasticity in spiking agents

Reward-modulated STDP was proposed to solve the distal-reward problem and to implement reinforcement learning in spiking networks [8, 9, 10], and Frémaux and Gerstner [11] place it in the general framework of three-factor learning rules (Section 2.3.2). Applications fall into two groups.

In vision, Mozafari et al. [?] train a spiking network for categorisation using first spikes and a reward signal, without an external classifier. In control, Bing et al. [23] use R-STDP synapses to learn lane keeping end to end on a Pioneer robot, with input from a neuromorphic vision sensor, and Yang et al. [24] derive an R-STDP method from the global policy gradient and apply it to vision-based navigation in ViZDoom and to robot-control tasks. These works show that R-STDP can learn closed-loop control, so the open question is not whether it can control an agent but under which conditions.

Two limitations recur. First, the network structure is chosen by the designer and fixed before learning. Second, as Yang et al. note, local rules often do not align with the global learning objective, which limits performance [24]. Evolving the structure while using R-STDP only for within-episode adaptation, as proposed here, is a different division of labour between the global and the local learning mechanism.

3.4 Evolved plastic networks

Evolving the learning process, and not only the weights, has a long history. Early work in evolutionary robotics evolved local synaptic-plasticity rules instead of fixed weights [25]. Soltoggio et al. evolved modulatory topologies, that is, networks that decide where neuromodulatory signals are routed, and showed that they outperform fixed-weight and Hebbian networks in a simulated-bee, reinforcement-learning-like task [26]. Later work studied the evolutionary advantages of neuromodulated plasticity in dynamic, reward-based scenarios [27], and the field has been surveyed under the name Evolved Plastic Artificial Neural Networks (EPANNs) [28].

A few works bring this idea to spiking networks. Lu et al. [29] use Cartesian genetic programming to evolve the mathematical expressions of plasticity rules for LIF networks and report rules that match or exceed R-STDP baselines on XOR and cart-pole; the network structure is fixed. Efe [30], in an MSc thesis, applies NEAT to spiking networks and reports that it solves XOR, pole balancing and food chasing but not problems that require adaptation during the agent's lifetime, which are solved after adding dopamine-modulated STDP. Pontes-Filho et al. [31] propose the NAGI framework, in which a modified NEAT evolves spiking controllers whose adaptation is coordinated by STDP, evaluated on foraging and logic-gate tasks.

These are the works closest to this project. Their tasks are small (XOR, cart-pole, foraging

in simple worlds), their inputs are not pixel-derived, and, to the best of our knowledge, none compares energy against a matched conventional baseline.

3.5 Energy-aware evaluation and neuromorphic hardware

Green AI [5] argues that computational cost should be reported alongside accuracy. At the hardware level, the energy of an operation depends strongly on its type: in the widely cited 45-nm estimates of Horowitz [?], a 32-bit floating-point multiply-accumulate costs several times more than an accumulate, and memory access costs more than arithmetic. Neuromorphic processors such as Loihi [32] implement event-driven spiking computation with on-chip learning; Davies et al. [33] survey results obtained on this platform and Schuman et al. [34] discuss opportunities and open problems for neuromorphic algorithms. Control-theoretic foundations for neuromorphic controllers are still being developed [?].

Energy claims for spiking networks are of two kinds: measurements on neuromorphic hardware, as in the PopSAN work above, and operation-count models evaluated on conventional hardware. The second kind is easy to apply, and it is the approach adopted in this project (Section 2.2.4), but it ignores memory and communication costs. Its results should therefore be read as comparative estimates between two models under explicit assumptions, not as predictions of power consumption on a given device.

3.6 Comparative analysis and research gap

Table 3.1 compares the works reviewed along the dimensions introduced at the start of the chapter.

Three patterns emerge from the table. First, gradient-free game-playing agents use conventional networks and do not analyse inference energy. Second, spiking controllers are either trained offline with global, gradient-based methods, or evolved without lifetime adaptation, whereas R-STDP controllers adapt through local rules but with a structure fixed by the designer. Third, the few works that combine evolution, spiking dynamics and plasticity are evaluated on small tasks, without pixel-derived input and without an energy comparison against a matched conventional baseline.

To the best of our knowledge, no existing work combines (i) NEAT-style structural evolution, (ii) LIF dynamics and (iii) R-STDP adaptation within an episode in a sequential platform game with pixel-derived input, and compares it with a matched NEAT baseline using an explicit inference-energy model. This project addresses that gap through the following questions, which the experiments of this work will answer:

Table 3.1: Positioning of related work. “n/r”: not reported in the cited work. “Local plasticity” means that weights change through local (STDP-type) rules; it does not indicate whether learning continues after deployment.

Work	Network	Learned or evolved	Local plasticity	Task	Energy
Mnih et al. [?]	ANN	Weights (backprop.)	No	Atari games	n/r
Hausknecht et al. [16]	ANN	Weights, topology (neuroevolution)	No	Atari games	n/r
MarI/O, NEAT [6?]	ANN	Topology, weights (NEAT)	No	Super Mario World	n/r
Qiu et al. [3]	SNN	Topology (evolved)	No	Nonlinear control	n/r
Pan et al. [?]	SNN	Architecture (evolutionary search)	No	Image classification	Considered
Tang et al. [22]	SNN actor	Weights (deep RL)	No	Continuous control	Measured (Loihi)
Hunter et al. [?]	SNN actor	Weights (SAC)	No	Continuous control	n/r
Mozafari et al. [?]	SNN	Weights (R-STDP)	Yes	Image categorisation	Qualitative
Bing et al. [23]	SNN	Weights (R-STDP)	Yes	Lane keeping (robot)	n/r
Yang et al. [24]	SNN	Weights (R-STDP)	Yes	ViZDoom, robot control	n/r
Soltoggio et al. [26, 27]	ANN	Modulatory topology, plasticity	Yes	Simulated foraging, dynamic reward tasks	n/r
Lu et al. [29]	SNN (LIF)	Plasticity rules (genetic programming)	Yes	XOR, cart-pole	n/r
Efe [30]	SNN	Topology (NEAT), dopamine-modulated STDP	Yes	XOR, pole balancing, foraging	n/r
Pontes-Filho et al. [31]	SNN	Modified NEAT, STDP	Yes	Foraging, logic gates	n/r
<i>This work</i>	SNN (LIF)	NEAT topology and initial weights; R-STDP within episodes	Yes	Super Mario Bros (pixel-derived input)	Estimated

1. Can a topology-evolved spiking controller with R-STDP reach at least 90% of the baseline performance, with no more than 10% additional completion time, on Level 1-1?
2. How does its estimated inference energy compare with that of the NEAT baseline under the operation-count model of Section 2.2.4?
3. How much does R-STDP contribute beyond evolution alone? This is examined through ablations (R-STDP disabled; Lamarckian write-back).

Conclusión

Bibliography

- [1] K. O. Stanley and R. Miikkulainen, “Evolving neural networks through augmenting topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.
- [3] H. Qiu, M. Garratt, D. Howard, and S. Anavatti, “Evolving spiking neural networks for nonlinear control problems,” in *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2018, pp. 1367–1373.
- [4] M. Mozafari, S. R. Kheradpisheh, T. Masquelier, A. Nowzari-Dalini, and M. Ganjtabesh, “First-spike-based visual categorization using reward-modulated STDP,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 12, pp. 6178–6190, 2018.
- [5] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [6] SethBling, “MarI/O - machine learning for video games,” <https://www.youtube.com/watch?v=qv6UVOQ0F44>, 2015, accessed: 2026-09-08.
- [7] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949.
- [8] E. M. Izhikevich, “Solving the distal reward problem through linkage of stdp and dopamine signaling in the brain,” *Cerebral Cortex*, vol. 17, no. 10, pp. 2443–2452, 2007.
- [9] R. V. Florian, “Reinforcement learning through modulation of spike-timing-dependent synaptic plasticity,” *Neural Computation*, vol. 19, no. 6, pp. 1468–1502, 2007.
- [10] R. Legenstein, D. Pecevski, and W. Maass, “A learning theory for reward-modulated spike-timing-dependent plasticity with application to biofeedback,” *PLoS Computational Biology*, vol. 4, no. 10, p. e1000180, 2008.

- [11] N. Frémaux and W. Gerstner, “Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules,” *Frontiers in Neural Circuits*, vol. 9, p. 85, 2016.
- [12] C. Kauten, “Super mario bros for openai gym,” <https://github.com/Kautenja/gym-super-mario-bros>, 2018, gitHub repository.
- [13] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*. IEEE, 2014, pp. 10–14.
- [14] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The Arcade Learning Environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.
- [15] S. Karakovskiy and J. Togelius, “The Mario AI benchmark and competitions,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 55–67, 2012.
- [16] M. Hausknecht, J. Lehman, R. Miikkulainen, and P. Stone, “A neuroevolution approach to general Atari game playing,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 4, pp. 355–366, 2014.
- [17] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, “Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning,” 2017.
- [18] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” 2017.
- [19] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, “Designing neural networks through neuroevolution,” *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.
- [20] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks,” *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [21] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, p. 3625, 2020.
- [22] G. Tang, N. Kumar, R. Yoo, and K. P. Michmizos, “Deep reinforcement learning with population-coded spiking neural network for continuous control,” in *Proceedings of the 2020 Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 155, 2021.

- [23] Z. Bing, C. Meschede, K. Huang, G. Chen, F. Röhrbein, M. Akl, and A. Knoll, “End to end learning of spiking neural network based on R-STDP for a lane keeping vehicle,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 4725–4732.
- [24] Z. Yang, S. Guo, Y. Fang, Z. Yu, and J. K. Liu, “Spiking variational policy gradient for brain inspired reinforcement learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 3, pp. 1975–1990, 2025.
- [25] D. Floreano and J. Urzelai, “Evolutionary robots with on-line self-organization and behavioral fitness,” *Neural Networks*, vol. 13, no. 4–5, pp. 431–443, 2000.
- [26] A. Soltoggio, P. Dürri, C. Mattiussi, and D. Floreano, “Evolving neuromodulatory topologies for reinforcement learning-like problems,” in *2007 IEEE Congress on Evolutionary Computation*, 2007.
- [27] A. Soltoggio, J. A. Bullinaria, C. Mattiussi, P. Durr, and D. Floreano, “Evolutionary advantages of neuromodulated plasticity in dynamic, reward-based scenarios,” *Proceedings of the 11th International Conference on Artificial Life*, pp. 569–576, 2008.
- [28] A. Soltoggio, K. O. Stanley, and S. Risi, “Born to learn: the inspiration, progress, and future of evolved plastic artificial neural networks,” *Neural Networks*, vol. 108, pp. 48–67, 2018.
- [29] J. Lu, J. J. Hagenaars, and G. C. H. E. de Croon, “Evolving-to-learn reinforcement learning tasks with spiking neural networks,” 2022.
- [30] A. Efe, “Evolutionary adaptation and dopamine modulated learning in spiking neural networks,” Master’s thesis, Bilkent University, 2021.
- [31] S. Pontes-Filho, K. Olsen, A. Yazidi, M. A. Riegler, P. Halvorsen, and S. Nichele, “Towards the neuroevolution of low-level artificial general intelligence,” *Frontiers in Robotics and AI*, vol. 9, p. 1007547, 2022.
- [32] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [33] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. F. Guerra, P. Joshi, P. Plank, and S. R. Risbud, “Advancing neuromorphic computing with Loihi: A survey of results and outlook,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021.
- [34] C. D. Schuman, S. R. Kulkarni, M. Parsa, J. P. Mitchell, P. Date, and B. Kay, “Opportunities for neuromorphic computing algorithms and applications,” *Nature Computational Science*, vol. 2, no. 1, pp. 10–19, 2022.